Entity Relationship (ER) Diagrams — In-Depth Notes

ER diagrams help us design how data should be organized and connected in a database. They are especially useful when an application has several related entities and the data must remain consistent over time.

In these notes, we will use a college course-enrollment system to understand entities, attributes, keys, relationships, cardinality, Crow's Foot notation, and the steps involved in database design.

---

1. Why Do We Need ER Diagrams?

Consider a college application where Rahul enrolls in a Java Programming course.

A Java application might create the following objects:

```
Student student = new Student(101, "Rahul");
Course course = new Course(201, "Java Programming", 30);
Enrollment enrollment = new Enrollment(
    student,
    course,
    LocalDateTime.now()
);
```

These objects exist in application memory. If the application restarts before the information has been saved, the enrollment data is lost.

A college system cannot depend on in-memory objects alone. It must store information about students, courses, and enrollments so that it remains available even when the application is restarted. A relational database such as MySQL can provide that persistent storage.

The next problem is **how to structure the stored data**. We need to decide:

- What information should be stored?

- Which pieces of information belong together?
- How should students, courses, and enrollments be connected?
- What rules should the database enforce to prevent invalid data?

ER modeling helps answer these questions before we start creating tables.

Separating the data into entities

For the enrollment system, there are three important entities:

- **Student:** A person registered in the college.
- **Course:** A course offered by the college.
- **Enrollment:** A record connecting a student to a course.

A student exists independently of any one enrollment. A course also exists independently of any one enrollment. Enrollment records the association between the two.

Students

student_id	name
101	Rahul
102	Priya
103	Aman

Courses

course_id	title	capacity
201	Java	30
202	Python	25

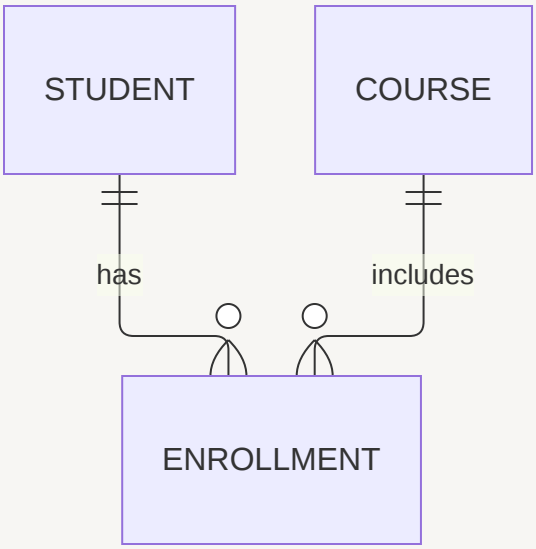
Enrollments

enrollment_id	student_id	course_id
1	101	201
2	101	202
3	102	201

enrollment_id	student_id	course_id
4	103	201

With this structure, Rahul's details are stored in the Students table, course details are stored in the Courses table, and the Enrollments table records their associations. The same student or course does not need to be fully duplicated in every enrollment record.

A basic ER diagram for this design looks like this:



This diagram shows that a student can have multiple enrollments and a course can have multiple enrollments. Each enrollment belongs to one student and one course.

---

2. What Is an Entity Relationship Diagram?

An **Entity Relationship (ER) diagram** is a visual representation of the data that a system needs to store and the relationships among that data.

An ER diagram typically describes four things:

1. **Entities:** The things about which we store data.
2. **Attributes:** The information stored for each entity.

3. **Relationships:** How the entities are associated.
4. **Cardinality and constraints:** How many records can participate in a relationship and which associations are required or optional.

ER diagrams are commonly used during relational database design.

ER diagrams and UML class diagrams are not the same. A UML class diagram models software classes, their properties, behavior, and associations. An ER diagram focuses on the structure and relationships of stored data. They can describe similar domain concepts, but they serve different purposes.

Two common ER diagram notations are **Chen notation** and **Crow's Foot notation**. These notes use Crow's Foot notation because it is widely used in practical database modeling.

---

3. Entities and Attributes

3.1 Entities

An **entity** is something in the domain about which the system needs to maintain information.

Examples in the college system include Student, Course, and Enrollment. In a relational implementation, an entity is commonly represented by a table.

For example:

```
CREATE TABLE students (
    student_id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(255)
);
```

The concept of a Student exists before we decide whether to use MySQL or what SQL data types to assign. The ER model captures the domain; the tables are an implementation of that model.

3.2 Attributes

An **attribute** describes a characteristic of an entity.

For a Student, attributes could be `student_id`, `name`, `email`, and `date_of_birth`.

For a Course, attributes could be `course_id`, `title`, and `capacity`.

Different kinds of attributes require different design decisions.

Simple attributes

A simple attribute is treated as a single meaningful value in the chosen model.

Examples include `student_id`, `course_id`, and `capacity`.

Composite attributes

A composite attribute can be divided into meaningful smaller parts.

For example, a student's name might be modeled as:

```
Name
├── First Name
└── Last Name
```

In a table, those parts could be stored separately:

```
first_name VARCHAR(50),
last_name VARCHAR(50)
```

Whether a name should be split depends on the application's requirements, such as searching, sorting, and displaying names.

Multivalued attributes

A multivalued attribute can have more than one value for the same entity. A student, for example, might register multiple phone numbers.

Storing them in a single column like this creates problems:

```
student_id = 101
phone_numbers = "9876543210, 9123456780"
```

It becomes harder to search, validate, and update one phone number independently.

A relational design can represent the values in a separate table:

student_phone_numbers

student_id	phone_number
101	9876543210
101	9123456780

In a conceptual ER model, phone number may be shown as a multivalued attribute. In the relational design, it is commonly converted into a separate table.

Derived attributes

A derived attribute is calculated from other stored information.

For example:

- **Age** can be calculated using `date_of_birth` and the current date.
- **Number of enrolled students** can be calculated by counting the relevant active enrollment records for a course.

Derived values do not always need to be stored. If they are stored separately, they can become outdated when the underlying data changes. Some applications cache calculated values for performance, but then they need a reliable way to keep those values consistent.

---

4. Primary Keys and Foreign Keys

4.1 Primary key (PK)

A **primary key** uniquely identifies each row in a table.

Suppose two students have the same name:

student_id	name
101	Rahul
105	Rahul

The name cannot reliably identify a student because names are not necessarily unique. The `student_id` can distinguish the two records.

```
CREATE TABLE students (
    student_id BIGINT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
```

A primary key must be **unique** and **not null**. In a practical ER diagram, it is commonly marked with `PK`.

4.2 Foreign key (FK)

A **foreign key** connects a column in one table to a key in another table.

For example, the Enrollments table contains `student_id`. That column refers to `student_id` in Students.

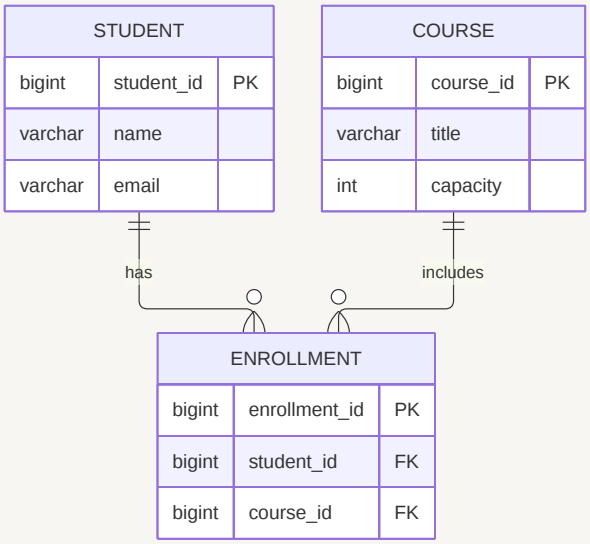
```
FOREIGN KEY (student_id)
REFERENCES students(student_id)
```

If an enrollment contains `student_id = 101`, the record refers to Rahul's student record.

A foreign key is used to enforce **referential integrity**. When the database enforces the constraint, an enrollment cannot refer to a student who does not exist. For instance, trying to create an enrollment for `student_id = 999` should fail if there is no student with that ID.

Without enforced foreign-key constraints, a database can accumulate **orphaned records**: records that refer to another record that is missing.

Showing the keys in an ER diagram



Here, `student_id` and `course_id` are primary keys in their own tables and foreign keys in the Enrollments table.

---

5. Relationships and Cardinality

A **relationship** describes how entities are associated. **Cardinality** specifies how many records of one entity can be associated with a record of another entity.

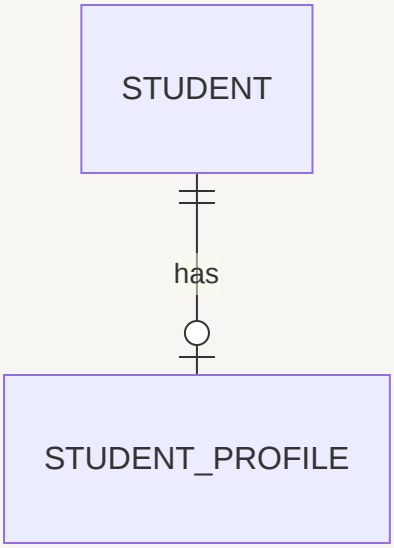
This is similar to multiplicity in UML class diagrams, but it is shown using ER modeling notation.

5.1 One-to-one (1:1)

Suppose a student can have at most one student profile, while every student profile must belong to exactly one student.

- A student may have **zero or one** profile.
- A student profile must have **exactly one** student.

A student may have no profile if the student has not completed it yet.



A possible SQL design places `student_id` inside `student_profiles`:

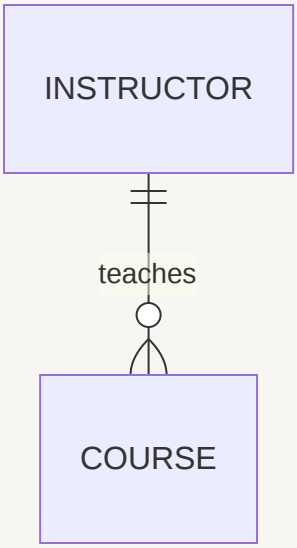
```
student_id BIGINT NOT NULL UNIQUE,
FOREIGN KEY (student_id) REFERENCES students(student_id)
```

The foreign key ensures the referenced student exists. The `UNIQUE` constraint ensures that the same student cannot have two profile rows. **A foreign key by itself does not enforce a one-to-one relationship.**

5.2 One-to-many (1:N)

Suppose an instructor can teach several courses, but each course is assigned to exactly one instructor.

- An instructor may teach **zero or many** courses.
- Each course must have **exactly one** instructor.



The `courses` table can store the instructor reference:

```
instructor_id BIGINT NOT NULL,
FOREIGN KEY (instructor_id) REFERENCES instructors(instructor
_id)
```

Several courses can share the same `instructor_id`.

In a typical one-to-many relational design, the foreign key is stored on the many side. Here, that is the Courses table.

5.3 Many-to-many (M:N)

In the enrollment system:

- One student can enroll in many courses.
- One course can have many students.

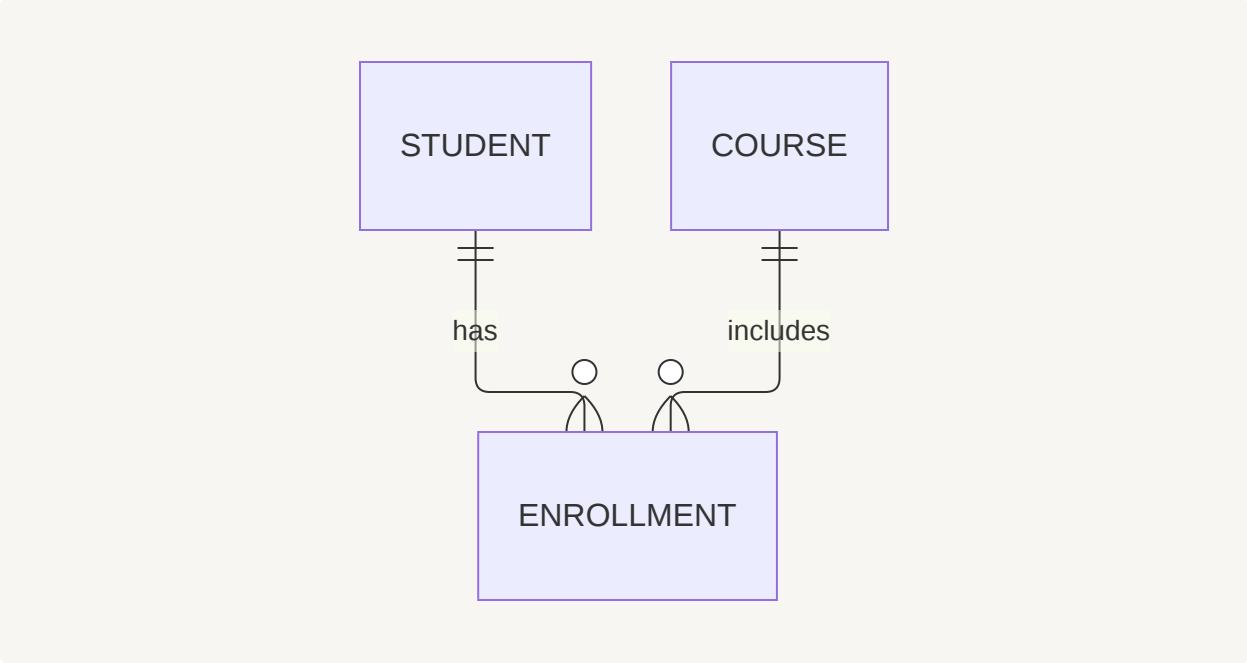
That is a many-to-many relationship.

A poor design would store a list of course IDs in one Students column:

```
student_id | course_ids
101        | 201, 202, 203
```

This makes individual associations difficult to query, validate, and constrain. Storing a list of student IDs in Courses would have the same problem.

The usual relational solution is an **associative entity**, also called a **junction table**. Here, the junction table is `enrollments`.



Each Enrollment row connects exactly one Student to exactly one Course. Together, the two one-to-many relationships implement the original many-to-many association.

---

6. Understanding Crow's Foot Notation

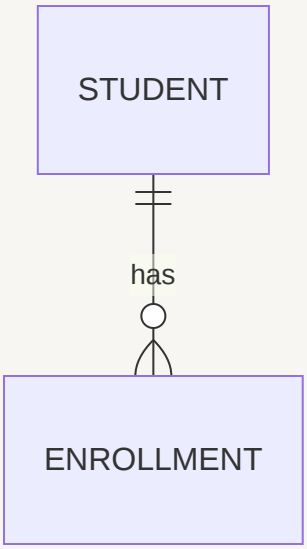
Crow's Foot notation shows the **minimum** and **maximum** number of related records at each end of a relationship.

Symbol	Meaning
`||`	Exactly one
`o|`	Zero or one
`|{`	One or many
`o{`	Zero or many

The symbols combine two ideas:

- **Minimum participation:** Is the relationship optional (`o`) or required (`|`)?
- **Maximum participation:** Can there be only one related record (`|`) or many (`{`)?

For example:



Read it from both directions:

- The `||` near **Student** means each Enrollment must be associated with **exactly one Student**.
- The `o{` near **Enrollment** means one Student may be associated with **zero or many Enrollments**.

Reading rule: Interpret a symbol at one end from the perspective of one record at the opposite end.

This is important because the two ends can communicate different constraints in the same relationship.

---

7. Designing an ER Diagram from Requirements

A systematic process makes database design easier to reason about.

Step 1: Understand the requirements

Identify the information the system must maintain and the rules it must enforce.

For our example:

- Students can enroll in courses.
- A student can enroll in multiple courses.
- A course can have multiple students.
- Each enrollment must identify its student and course.

Step 2: Identify the entities

Identify the main concepts that need to be represented independently.

For this system: `Student`, `Course`, and `Enrollment`.

Step 3: Identify the attributes

List the information that belongs to each entity.

- **Student:** student ID, name, email.
- **Course:** course ID, title, capacity.
- **Enrollment:** enrollment ID, student ID, course ID.

Step 4: Identify relationships and constraints

Determine how entities connect and how many records can participate.

- Student → Enrollment: one-to-many.
- Course → Enrollment: one-to-many.
- Each Enrollment → Student: exactly one.
- Each Enrollment → Course: exactly one.

Add the relevant primary and foreign keys.

Step 5: Convert the design into SQL tables

Once the entities, attributes, and relationships are clear, create the relational tables and enforce the required constraints in the database.

---

8. Conceptual, Logical, and Physical Data Models

ER modeling can be developed at different levels of detail.

Model	Main focus	Example
Conceptual	Main entities and their relationships, without implementation details	Student enrolls in Course
Logical	Attributes, identifiers, relationships, and data structure	Student, Course, and Enrollment entities with their keys
Physical	Actual tables, columns, SQL data types, and database constraints	MySQL `CREATE TABLE` statements

The distinction is useful because database design begins with business concepts, not SQL syntax.

Conceptual model

At this level, the system can be described simply as **students enroll in courses**. The aim is to understand the domain and the relationships.

Logical model

The model becomes more detailed. We identify attributes and keys and decide that Enrollment is needed to represent the many-to-many relationship in a relational database. Normalization may be applied at this stage to reduce repeated data and improve consistency.

Physical model

The design is translated into concrete database objects with SQL types and constraints.

For example, the Enrollments table could be defined as:

```
CREATE TABLE enrollments (
    enrollment_id BIGINT PRIMARY KEY,
    student_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
```

```
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

The two `NOT NULL` foreign-key columns reflect the rule that every enrollment must refer to one student and one course.

---

9. Key Takeaways

- ER diagrams model the information a system needs to store and the relationships among that information.
- Entities represent important domain concepts; attributes describe those entities.
- Primary keys uniquely identify rows, while foreign keys connect related tables and help enforce referential integrity.
- The three common relationship types are one-to-one, one-to-many, and many-to-many.
- Many-to-many relationships are commonly implemented using a junction table such as Enrollment.
- Crow's Foot symbols show both minimum and maximum cardinality.
- Conceptual models describe the domain, logical models organize the data structure, and physical models specify actual database implementation details.

Design principle: Understand the data and its relationships before creating the SQL tables. The database schema should reflect the rules of the application, not just the objects currently present in memory.